



Санкт-Петербургский государственный университет
Кафедра системного программирования

Валидация спецификации RISC-V ISA в LLVM относительно Sail

Хечнев Семен Евгеньевич, группа 21.Б07-мм

Научный руководитель: инженер-исследователь лаборатории технологий программирования инфраструктурных решений, Косарев Д. С.

Санкт-Петербург
2025

- RISC-V — открытая, расширяемая ISA (Instruction Set Architecture)
- LLVM¹ — инфраструктура для компиляторов
 - ▶ Спецификация ISA описывается декларативно на TableGen
- Sail² — язык для описания моделей процессорных архитектур. Позволяет задать семантику исполнения процессорных инструкций
 - ▶ sail-riscv³ — формальная, поведенческая спецификация RISC-V ISA
- Актуальна проблема валидации спецификаций относительно друг друга

¹<https://llvm.org>

²<https://github.com/rem-s-project/sail/>

³<https://github.com/riscv/sail-riscv>

Постановка задачи

Целью работы является валидация спецификации RISC-V ISA в LLVM относительно Sail

Задачи:

- Разработать прототип анализатора, который будет включать следующие анализы
 - ▶ Поиск входных и выходных операндов инструкций
 - ▶ Определение того, взаимодействует ли инструкция с памятью
- При помощи анализатора сравнить следующие спецификации RISC-V ISA:
 - ▶ LLVM v19 против sail-riscv⁴
 - ▶ P extension (version 0.9.11-draft-20211209) в LLVM против sail-riscv-p⁵
- Протестировать анализатор

⁴<https://github.com/riscv/sail-riscv/commits/05b845c91d1c1db7b361fc8d06e815b54ca0b07a>

⁵<https://github.com/umcann123/sail-riscv-p>

- TableGen⁶ — декларативный язык
- Используется для описания спецификации ISA целевой платформы
 - ▶ входные, выходные операнды
 - ▶ читает из памяти, пишет в память
 - ▶ ...

Listing 1: Описание инструкции `addi` на TableGen

```
let isAsCheapAsAMove = 1 in
def ADDI : RVInstI< ... , (outs GPR:$rd),
                (ins GPR:$rs1, simm12:$imm12)
                "addi", "$rd, $rs1, $imm12">
```

⁶<https://llvm.org/docs/TableGen/>

- Sail — язык для описания моделей процессорных архитектур
- Модели
 - ▶ ARM
 - ▶ X86
 - ▶ RISC-V
- Позволяет генерировать эмуляторы процессорных архитектур
- Верифицировать модели
- Генерировать и исполнять автоматические тесты
- Исполнять тесты безопасности кода

Реализация модели архитектуры RISC-V на Sail состоит из:

- AST, причём один узел может представлять множество инструкций
- `encdec` функции, задающие отображения `opcode` в узел AST
- `execute` функции, задающие семантику исполнения
- `assembly` функции, отображающие узлы AST в строки

```
function clause execute (ADDI(imm, rs1, rd)) = {  
    let rs1_val = X(rs1);           // read the rs1  
    let immext : xlenbits = EXTS(imm); // sign-extend imm  
    let result : xlenbits = rs1_val + immext; // compute  
    X(rd) = result;                 // write result to rd  
    RETIRE_SUCCESS  
}
```

- Для поиска входных и выходных операндов строится граф вызовов для всех `execute` функций
- Производится обход этого графа и определение того, какие функции с какими аргументами вызывали функции для чтения/записи регистров
 - ▶ Дополнительно для каждой функции производился анализ алиасов

Реализация. Обращения инструкции к памяти

- Для работы с памятью в sail-riscv используются функции `read_ram` и `write_ram`
- Для того чтобы определить работает ли инструкция с памятью, для каждой функции `execute` проверялась достижимость вершин `read_ram` и `write_ram` в графе вызовов всех функций
- Однако любой инструкции, работающей с адресами, необходимо транслировать виртуальный адрес в физический
 - ▶ Чтение из памяти могло происходить в случае, если при трансляции адреса происходил TLB miss. Тогда необходимо было прочитать таблицу виртуальных адресов из памяти
 - ▶ Запись в память могла происходить тогда, когда при трансляции адреса необходимо было обновить специальные биты (A или D) у записи в таблице виртуальных адресов

Реализация. Особенности sail-riscv-p

- Sail-riscv-p⁷ — это форк sail-riscv с реализацией расширения P (packed SIMD)
- Для RV32 регистры могут интерпретироваться как пара

```
function clause execute(PEXT_64BITS_ADD_SUB(rs2 , rs1 , rd , op))  
...  
    rs1_val = X(rs1 | 0b00001) @ X(rs1 & 0b11110);  
...
```

- 1 Модель представлялась в ANF форме
- 2 Определялись входные и выходные операнды по графу вызовов
- 3 Построение и анализ data flow графа — определение зависимости операндов найденных на предыдущем шаге

```
let a1 = rs1 | 0b00001 in  
let a2 = rs1 & 0b11110 in  
rs1_val = X(a1) @ X(a2);
```

⁷<https://github.com/umcann123/sail-riscv-p>

- Расширение C
 - ▶ Для 10 инструкций SP не является явным аргументом в Sail
 - ▶ Для 2 инструкций RA не является выходным операндом в LLVM
- Расширение V
 - ▶ Для 26 инструкций vd не является входным операндом в LLVM
 - ▶ Для 277 инструкций vd и v0 не является входным операндом в LLVM
- Расширение P (sail-riscv-p)
 - ▶ Для 40 инструкций rd не является входным операндом в LLVM

Таким образом, в рамках преддипломной практики были достигнуты следующие результаты:⁸

- разработан прототип анализатора, который реализует следующие анализы:
 - ▶ поиск входных и выходных операндов инструкций
 - ▶ определение, обращается ли инструкция к памяти
- при помощи анализатора сравнены следующие спецификации RISC-V ISA:
 - ▶ LLVM v19 против sail-riscv
 - ▶ P extension в LLVM против sail-riscv-p

⁸Исходный код — https://github.com/s-khechnev/llvm_td_lint_playground/blob/ins_outs_regs
Имя коммитера: s-khechnev

Listing 2: Пример функции, которая интерпретирует выполнение инструкции c.addi4spn

```
function clause execute (C_ADDI4SPN(rdc, nzimm)) = {  
    let imm : bits(12) = (0b00 @ nzimm @ 0b00);  
    let rd = creg2reg_idx(rdc);  
    execute(ITYPE(imm, sp, rd, RISCV_ADDI))  
}
```